 # USART Serial Communication System

Overview

This project implements a bare-metal Universal Synchronous/Asynchronous Receiver-Transmitter (USART) driver. It is responsible for bridging the STM32 to a PC, providing real-time serial logging of the environmental data captured by the DHT11 sensor.

Objectives

*** Develop a custom USART driver to handle asynchronous serial framing.**

*** Automatically calculate and configure Baud Rate mantissa and fraction registers.**

*** Implement hardware flag polling (`TXE`) to safely transmit character arrays.**

*** Format raw integer data into human-readable text strings using standard C libraries.**

Components Used

*** STM32 Nucleo Board (STM32F401RE / F446RE)**

*** ST-Link Virtual COM Port**

*** PC Serial Terminal (e.g., PuTTY, TeraTerm)**

Working Principle

USART relies on an agreed-upon clock speed (baud rate) rather than a shared physical clock line. The driver takes the APB bus frequency and the desired baud rate (9600) to compute the fractional clock divider. Data is loaded into the `USART_DR` register, framed with a start bit and stop bit, and pushed down the TX line.

System Architecture

*** **GPIO Alternate Functions:** PA2 (TX) and PA3 (RX) mapped to AF7.**

*** **Transmission Logic:** A `while` loop that iterates over a null-terminated string, waiting for the Transmit Data Register Empty (`TXE`) flag to go HIGH before sending the next byte.**

Implementation Flow

1. Enable USART2 and GPIOA peripheral clocks.

2. Configure PA2 and PA3 to Alternate Function 7 (AF7).

3. Initialize USART2 with 9600 Baud, 8 Data Bits, 1 Stop Bit, and No Parity.

4. Read DHT11 data and format it using `sprintf()`.

5. Call `USART_Transmit_String()` to log the data to the PC.

Key Observations

* **Flag Management:** Attempting to write to the Data Register before `TXE` is set results in dropped characters and corrupted strings.

* **Formatting:** Abstracting the raw data into formatted strings (`"Temp: 32.5 C"`) is critical for debugging and readability.

Results

A highly stable serial connection outputting continuous, easily readable environmental data to the PC monitor without character dropping or buffer overflows.

Example Output:

`Temp: 32.5 C`

Key Learnings

* Understanding how fractional baud rate generation translates to physical register configurations.

* Hiding the complexity of `TXE` and `TC` flag polling inside a clean `Transmit_String` API.

💻 **Source Code:** https://github.com/adlernunez/Adler-Entri/tree/master/ARM%20F401xx/FINAL%20PROJECT/STM32F446RE

Application Layer (main.c snippet)

```c
#include "stm32f401re_usart_driver.h"
#include <stdio.h>

int main(void) {
    // Initialize USART2 (PA2 TX, PA3 RX)
    usart2_GPIO_PinSetUp();
    USART_Handle_t usart2 = {0};
    usart2.pUSARTx = USART2;
```

```
  usart2.USART_Config.USART_Baud = USART_STD_BAUD_9600;
  usart2.USART_Config.USART_Mode = USART_MODE_TX_ONLY;
  usart2.USART_Config.USART_Parity = USART_NO_PARITY;
  usart2.USART_Config.USART_StopBit = USART_STOP_BIT_1;
  usart2.USART_Config.USART_WordLength = USART_WORDLEN_8BITS;
  USART_Init(&usart2);
  USART_PeripheralControl(USART2, ENABLE);

  char display_buffer[32];
  int temp = 32;

  while(1) {
    sprintf(display_buffer, "Temp: %d C \r\n", temp);
    USART_Transmit_String(&usart2, display_buffer);
    delay_ms(2000);
  }
}
```

DEMO:https://drive.google.com/drive/folders/1eXq7sDkj5dFtnOcIj5tX9Nme-Wcqtehi?usp=sharing